

CS543 – Web Services

Assignment 4: REST Service, OpenAPI Contract and Reliable Integration

Selected Service: Orders (CampusEats)

Team ID: 10

Team Leader: Annu Kumari — 20251651024

Team Members: Prerna — 20251651072; Akanksha Upadhaya — 20251651011

1. Objective

Rebuild the selected CampusEats service as a REST service, model resources using HTTP semantics, publish an OpenAPI contract, implement at least four endpoints, use correct status codes and a common problem response shape, support safe retries through Idempotency-Key, and harden the outbound Payments call with timeout, retry and fallback behaviour.

2. REST Resource Design

The Orders service models orders as a durable plural resource. It provides create/list, single-order retrieval, filtered listing, and cancellation as a sub-resource.

Method	URL	Purpose	Success	Failure
POST	/orders	Create an order and charge payment	201 + Location	400 / 422 / 503
GET	/orders/{id}	Read one order	200	404
GET	/orders?userId=&status=	Filtered list	200	400
POST	/orders/{id}/cancellation	Request cancellation	202	404 / 409

3. OpenAPI and Implementation

The project contains openapi.yaml, models.py, store.py, errors.py, payments_client.py, app.py, tests/test_orders.py, NOTES.md and supporting files. Schemas are defined once under components.schemas and referenced with \$ref. The common problem() helper returns type, title, status and detail.

4. Idempotency and Network Hardening

The create endpoint accepts Idempotency-Key and stores it with the order. A repeated key returns the original result without repeating payment work. The Payments client uses an environment-configured URL, timeout, exponential backoff with jitter and safe retry handling. If Payments is unreachable, the service returns 503 rather than creating an unconfirmed order.

5. Execution Evidence – Screenshots Provided

OpenAPI Validation and Pytest

```
-3.19 iniconfig-2.3.0 itsdangerous-2.2.0 jsonschema-4.26.0 jsonschema-path-0.3.4 jsonschema-specifications-2025.9.1 lazy-object-proxy-1.12.0 openapi-schema-validator-0.6.3 openapi-spec-validator-0.7.1 packaging-26.3 pathable-0.4.4 pluggy-1.6.0 pytest-8.3.4 referencing-0.36.2 requests-2.32.3 rfc3339-validator-0.1.4 rpds-py-2026.6.3 six-1.17.0 urllib3-2.7.0
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> python -c "import flask; print('Flask OK')"
Flask OK
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> openapi-spec-validator openapi.yaml
openapi.yaml: OK
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> pytest -q
.... [100%]
4 passed in 0.67s
```

Figure 1: openapi.yaml: OK and 4 passed in 0.67s.

Flask Service Running

```
4 passed in 0.67s [100%]
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> flask --app app run --port 8090
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8090
Press CTRL+C to quit
127.0.0.1 - - [08/Sep/2026 20:25:15] "GET /orders/999 HTTP/1.1" 404 -
127.0.0.1 - - [08/Sep/2026 20:26:19] "POST /orders HTTP/1.1" 400 -
127.0.0.1 - - [08/Sep/2026 20:27:40] "POST /orders HTTP/1.1" 503 -
[]
```

Figure 2: Flask Orders service running on port 8090.

Unknown Order – 404

```
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> curl.exe -i http://127.0.0.1:8090/orders/999
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.1.8 Python/3.13.15
Date: Tue, 08 Sep 2026 14:55:15 GMT
Content-Type: application/json
Content-Length: 98
Connection: close
```

Figure 3: GET /orders/999 returns HTTP 404 NOT FOUND.

Payments Failure – 503

```
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> curl.exe -i -X POST "http://127.0.0.1:8090/orders" -H "Content-Type: application/json" -H "Idempotency-Key: order-001" --data-raw '{"userId":10,"itemId":101,"quantity":2,"amount":500,"cardToken":"demo-token"}'
HTTP/1.1 503 SERVICE UNAVAILABLE
Server: Werkzeug/3.1.8 Python/3.13.15
Date: Tue, 08 Sep 2026 14:57:40 GMT
Content-Type: application/json
Content-Length: 141
Connection: close

{"detail":"Payments service could not be reached","status":503,"title":"Payments service unavailable","type":"/errors/payments-unavailable"}
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders>
```

Figure 4: Create request returns HTTP 503 when Payments is unavailable.

Malformed Request – 400

```
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> curl.exe -i -X POST http://127.0.0.1:8090/orders `
>> -H "Content-Type: application/json" `
>> -H "Idempotency-Key: order-001" `
>> -d '{"userId":10,"itemId":101,"quantity":2,"amount":500,"cardToken":"demo-token"}'
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.8 Python/3.13.15
Date: Tue, 08 Sep 2026 14:56:19 GMT
Content-Type: application/json
Content-Length: 252
Connection: close

{"detail": "[['userId', 'required integer'], ['itemId', 'required integer'], ['quantity', 'positive integer'], ['amount', 'positive integer in parse'], ['cardToken', 'required']]","status":400,"title":"Invalid request","type":"/errors/invalid-request"}
curl: (3) URL rejected: Port number was not a decimal number between 0 and 65535
```

Figure 5: Invalid order request returns HTTP 400 BAD REQUEST.

6. Additional Execution Evidence

The additional execution screenshots supplied with this corrected report now demonstrate the required successful create, idempotency retry, successful single-order read, and 409 state-conflict behaviour. A separate filtered-list screenshot is not included in the provided evidence, so that item is not claimed as completed.

6.1 Successful Create – 201 + Location

```
Press CTRL+C to quit
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> $env:PAYMENTS_URL="http://127.0.0.1:8080"
>> flask --app app run --port 8090
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8090
Press CTRL+C to quit
127.0.0.1 - - [08/Sep/2026 21:21:49] "POST /orders HTTP/1.1" 201 -
127.0.0.1 - - [08/Sep/2026 21:22:33] "POST /orders HTTP/1.1" 200 -
127.0.0.1 - - [08/Sep/2026 21:23:25] "GET /orders/1 HTTP/1.1" 200 -
127.0.0.1 - - [08/Sep/2026 21:39:30] "POST /orders/1/cancellation HTTP/1.1" 202 -
127.0.0.1 - - [08/Sep/2026 21:39:45] "POST /orders/1/cancellation HTTP/1.1" 409 -
□
```

Figure 6: Successful POST /orders returns HTTP 201 CREATED with a Location header.

6.2 Idempotency Repeat

```
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> curl.exe -i -X POST "http://127.0.0.1:8090/orders" -H "Content-Type: application/json" -H "Idempotency-Key: order-004" --data-raw '{"userId":10,"itemId":101,"quantity":2,"amount":500,"cardToken":"demo-token"}'
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.13.15
Date: Tue, 08 Sep 2026 15:52:33 GMT
Content-Type: application/json
Content-Length: 142
Connection: close

{"amount":500,"createdAt":"2026-09-08T15:51:49.387885+00:00","id":1,"itemId":101,"paymentId":3,"quantity":2,"status":"confirmed","userId":10}
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> □
```

Figure 7: Repeated POST /orders using the same Idempotency-Key returns the original order result without creating a second order.

6.3 Successful Read – 200

```

PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> curl.exe -i http://127.0.0.1:8090/orders/1
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.13.15
Date: Tue, 08 Sep 2026 15:53:25 GMT
Content-Type: application/json
Content-Length: 142
Connection: close

{"amount":500,"createdAt":"2026-09-08T15:51:49.387885+00:00","id":1,"itemId":101,"paymentId":3,"quantity":2,"status":"confirmed","userId":10}
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders>

```

Figure 8: GET /orders/1 returns HTTP 200 OK with the order representation.

6.4 Cancellation State Conflict – 409

```

PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> curl.exe -i -X POST "http://127.0.0.1:8090/orders/1/cancellation"
HTTP/1.1 202 ACCEPTED
Server: Werkzeug/3.1.8 Python/3.13.15
Date: Tue, 08 Sep 2026 16:09:30 GMT
Content-Type: application/json
Content-Length: 35
Connection: close

{"orderId":1,"status":"cancelled"}
PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> curl.exe -i -X POST "http://127.0.0.1:8090/orders/1/cancellation"
HTTP/1.1 409 CONFLICT
Server: Werkzeug/3.1.8 Python/3.13.15
Date: Tue, 08 Sep 2026 16:09:45 GMT
Content-Type: application/json
Content-Length: 116
Connection: close

{"detail":"Order is already cancelled","status":409,"title":"Order state conflict","type":"/error"}

```

Figure 9: First cancellation returns HTTP 202 ACCEPTED; repeating cancellation returns HTTP 409 CONFLICT with an order-state conflict.

6.5 Supporting Service Log

```

PS C:\Users\Dell\Downloads\Assignment_4_Orders_Service\assignment4_orders> curl.exe -i -X POST "http://127.0.0.1:8090/orders" -H "Content-Type: application/json" -H "Idempotency-Key: order-001" --data-raw '{"userId":10,"itemId":101,"quantity":2,"amount":500,"cardToken":"demo-token"}'
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.8 Python/3.13.15
Date: Tue, 08 Sep 2026 15:33:58 GMT
Content-Type: application/json
Content-Length: 145
Location: /orders/1
Connection: close

{"amount":500,"createdAt":"2026-09-08T15:33:58.088676+00:00","id":1,"itemId":101,"paymentId":null,"quantity":2,"status":"confirmed","userId":10}
PS C:\Users> curl.exe -i -X POST "http://127.0.0.1:8090/orders" -H "Content-Type: application/json" -H "Idempotency-Key: order-003" --data-raw '{"userId":10,"itemId":101,"quantity":2,"amount":500,"cardToken":"demo-token"}'
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.8 Python/3.13.15
Date: Tue, 08 Sep 2026 15:46:18 GMT
Content-Type: application/json

```

Figure 10: Flask service log confirms POST /orders returned 201, GET /orders/1 returned 200, and the cancellation sequence returned 202 followed by 409.

7. Assignment 3 Comparison

Assignment 3 used the CampusPay SOAP boundary with WSDL, XSD messages and SOAP Faults. Assignment 4 maps durable business concepts to REST resources, HTTP methods/status codes, OpenAPI schemas and a common problem response. The external payment boundary remains an integration dependency.